JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Class ConcurrentLinkedDeque<E>

java.lang.Object
  java.util.AbstractCollection<E>
    java.util.concurrent.ConcurrentLinkedDeque<E>

Type Parameters:

`E - the type of elements held in this collection`

All Implemented Interfaces:

`Serializable, Iterable<E>, Collection<E>, Deque<E>, Queue<E>`

```
public class ConcurrentLinkedDeque<E>
extends AbstractCollection<E>
implements Deque<E>, Serializable
```

An unbounded concurrent deque based on linked nodes. Concurrent insertion, removal, and access operations execute safely across multiple threads. A `ConcurrentLinkedDeque` is an appropriate choice when many threads will share access to a common collection. Like most other concurrent collection implementations, this class does not permit the use of `null` elements.

Iterators and spliterators are *weakly consistent*.

Beware that, unlike in most collections, the `size` method is *NOT* a constant-time operation. Because of the asynchronous nature of these deques, determining the current number of elements requires a traversal of the elements, and so may report inaccurate results if this collection is modified during traversal. Additionally, the bulk operations `addAll`, `removeAll`, `retainAll`, `containsAll`, `equals`, and `toArray` are *not* guaranteed to be performed atomically. For example, an iterator operating concurrently with an `addAll` operation might view only some of the added elements.

This class and its iterator implement all of the *optional* methods of the `Deque` and `Iterator` interfaces.

Memory consistency effects: As with other concurrent collections, actions in a thread prior to placing an object into a `ConcurrentLinkedDeque` *happen-before* actions subsequent to the access or removal of that element from the `ConcurrentLinkedDeque` in another thread.

This class is a member of the Java Collections Framework.

Since:

1.7

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`ConcurrentLinkedDeque()`	Constructs an empty deque.
`ConcurrentLinkedDeque(Collection<? extends E> c)`	Constructs a deque initially containing the elements of the given collection, added in traversal order of the collection's iterator.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
boolean	`add(E e)`	Inserts the specified element at the tail of this deque.
boolean	`addAll(Collection<? extends E> c)`	Appends all of the elements in the specified collection to the end of this deque, in the order that they are returned by the

		specified collection's iterator.
void	**addFirst**(**E** e)	Inserts the specified element at the front of this deque.
void	**addLast**(**E** e)	Inserts the specified element at the end of this deque.
void	**clear**()	Removes all of the elements from this deque.
boolean	**contains**(**Object** o)	Returns `true` if this deque contains at least one element `e` such that `o.equals(e)`.
Iterator<E>	**descendingIterator**()	Returns an iterator over the elements in this deque in reverse sequential order.
E	**element**()	Retrieves, but does not remove, the head of the queue represented by this deque (in other words, the first element of this deque).
E	**getFirst**()	Retrieves, but does not remove, the first element of this deque.
E	**getLast**()	Retrieves, but does not remove, the last element of this deque.
boolean	**isEmpty**()	Returns `true` if this collection contains no elements.
Iterator<E>	**iterator**()	Returns an iterator over the elements in this deque in proper sequence.
boolean	**offer**(**E** e)	

		Inserts the specified element at the tail of this deque.
boolean	**offerFirst**(**E** e)	Inserts the specified element at the front of this deque.
boolean	**offerLast**(**E** e)	Inserts the specified element at the end of this deque.
E	**peek**()	Retrieves, but does not remove, the head of the queue represented by this deque (in other words, the first element of this deque), or returns `null` if this deque is empty.
E	**peekFirst**()	Retrieves, but does not remove, the first element of this deque, or returns `null` if this deque is empty.
E	**peekLast**()	Retrieves, but does not remove, the last element of this deque, or returns `null` if this deque is empty.
E	**poll**()	Retrieves and removes the head of the queue represented by this deque (in other words, the first element of this deque), or returns `null` if this deque is empty.
E	**pollFirst**()	Retrieves and removes the first element of this deque, or returns `null` if this deque is empty.
E	**pollLast**()	Retrieves and removes the last element of this

		deque, or returns `null` if this deque is empty.
E	**pop**()	Pops an element from the stack represented by this deque.
void	**push**(**E** e)	Pushes an element onto the stack represented by this deque (in other words, at the head of this deque) if it is possible to do so immediately without violating capacity restrictions, throwing an `IllegalStateException` if no space is currently available.
E	**remove**()	Retrieves and removes the head of the queue represented by this deque (in other words, the first element of this deque).
boolean	**remove**(**Object** o)	Removes the first element e such that `o.equals(e)`, if such an element exists in this deque.
E	**removeFirst**()	Retrieves and removes the first element of this deque.
boolean	**removeFirstOccurrence**(**Object** o)	Removes the first element e such that `o.equals(e)`, if such an element exists in this deque.
E	**removeLast**()	Retrieves and removes the last element of this deque.
boolean	**removeLastOccurrence**(**Object** o)	Removes the last element e such that

		`o.equals(e)`, if such an element exists in this deque.
`int`	**`size`**`()`	Returns the number of elements in this deque.
`Spliterator``<E>`	**`spliterator`**`()`	Returns a **`Spliterator`** over the elements in this deque.
`Object``[]`	**`toArray`**`()`	Returns an array containing all of the elements in this deque, in proper sequence (from first to last element).
`<T> T[]`	**`toArray`**`(T[] a)`	Returns an array containing all of the elements in this deque, in proper sequence (from first to last element); the runtime type of the returned array is that of the specified array.

Methods inherited from class java.util.AbstractCollection

`containsAll`, `removeAll`, `retainAll`, `toString`

Methods inherited from class java.lang.Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Methods inherited from interface java.util.Collection

`containsAll`, `equals`, `hashCode`, `parallelStream`, `removeAll`, `removeIf`, `retainAll`, `stream`

Methods inherited from interface java.lang.Iterable

`forEach`

Constructor Detail

ConcurrentLinkedDeque

```
public ConcurrentLinkedDeque()
```

Constructs an empty deque.

ConcurrentLinkedDeque

```
public ConcurrentLinkedDeque(Collection<? extends E> c)
```

Constructs a deque initially containing the elements of the given collection, added in traversal order of the collection's iterator.

Parameters:

`c - the collection of elements to initially contain`

Throws:

`NullPointerException - if the specified collection or any of its elements are null`

Method Detail

addFirst

```
public void addFirst(E e)
```

Inserts the specified element at the front of this deque. As the deque is unbounded, this method will never throw `IllegalStateException`.

Specified by:

`addFirst in interface Deque<E>`

Parameters:

`e - the element to add`

Throws:

`NullPointerException - if the specified element is null`

addLast

```
public void addLast(E e)
```

Inserts the specified element at the end of this deque. As the deque is unbounded, this method will never throw `IllegalStateException`.

This method is equivalent to `add(E)`.

Specified by:

`addLast` in interface `Deque<E>`

Parameters:

`e` - the element to add

Throws:

`NullPointerException` - if the specified element is null

offerFirst

```
public boolean offerFirst(E e)
```

Inserts the specified element at the front of this deque. As the deque is unbounded, this method will never return `false`.

Specified by:

`offerFirst` in interface `Deque<E>`

Parameters:

`e` - the element to add

Returns:

`true` (as specified by `Deque.offerFirst(E)`)

Throws:

`NullPointerException` - if the specified element is null

offerLast

```
public boolean offerLast(E e)
```

Inserts the specified element at the end of this deque. As the deque is unbounded, this method will never return `false`.

This method is equivalent to `add(E)`.

Specified by:

`offerLast` in interface `Deque<E>`

Parameters:

`e` - the element to add

Returns:

`true` (as specified by `Deque.offerLast(E)`)

Throws:

`NullPointerException` - if the specified element is null

peekFirst

```
public E peekFirst()
```

Description copied from interface: Deque

Retrieves, but does not remove, the first element of this deque, or returns `null` if this deque is empty.

Specified by:

`peekFirst` in interface `Deque<E>`

Returns:

the head of this deque, or null if this deque is empty

peekLast

```
public E peekLast()
```

Description copied from interface: Deque

Retrieves, but does not remove, the last element of this deque, or returns `null` if this deque is empty.

Specified by:

`peekLast` in interface `Deque<E>`

Returns:

the tail of this deque, or null if this deque is empty

getFirst

```
public E getFirst()
```

Description copied from interface: Deque

Retrieves, but does not remove, the first element of this deque. This method differs from peekFirst only in that it throws an exception if this deque is empty.

Specified by:

getFirst in interface Deque<E>

Returns:

the head of this deque

Throws:

NoSuchElementException - if this deque is empty

getLast

```
public E getLast()
```

Description copied from interface: Deque

Retrieves, but does not remove, the last element of this deque. This method differs from peekLast only in that it throws an exception if this deque is empty.

Specified by:

getLast in interface Deque<E>

Returns:

the tail of this deque

Throws:

NoSuchElementException - if this deque is empty

pollFirst

```
public E pollFirst()
```

Description copied from interface: Deque

Retrieves and removes the first element of this deque, or returns `null` if this deque is empty.

Specified by:

`pollFirst` in interface `Deque<E>`

Returns:

the head of this deque, or null if this deque is empty

pollLast

```
public E pollLast()
```

Description copied from interface: Deque

Retrieves and removes the last element of this deque, or returns `null` if this deque is empty.

Specified by:

`pollLast` in interface `Deque<E>`

Returns:

the tail of this deque, or null if this deque is empty

removeFirst

```
public E removeFirst()
```

Description copied from interface: Deque

Retrieves and removes the first element of this deque. This method differs from `pollFirst` only in that it throws an exception if this deque is empty.

Specified by:

`removeFirst` in interface `Deque<E>`

Returns:

the head of this deque

Throws:

`NoSuchElementException` - if this deque is empty

removeLast

```
public E removeLast()
```

Description copied from interface: Deque

Retrieves and removes the last element of this deque. This method differs from `pollLast` only in that it throws an exception if this deque is empty.

Specified by:

`removeLast` in interface `Deque<E>`

Returns:

the tail of this deque

Throws:

`NoSuchElementException` - if this deque is empty

offer

```
public boolean offer(E e)
```

Inserts the specified element at the tail of this deque. As the deque is unbounded, this method will never return `false`.

Specified by:

`offer` in interface `Deque<E>`

Specified by:

`offer` in interface `Queue<E>`

Parameters:

`e` - the element to add

Returns:

true (as specified by `Queue.offer(E)`)

Throws:

`NullPointerException` - if the specified element is null

add

```
public boolean add(E e)
```

Inserts the specified element at the tail of this deque. As the deque is unbounded, this method will never throw `IllegalStateException` or return `false`.

Specified by:

`add` in interface `Collection<E>`

Specified by:

`add` in interface `Deque<E>`

Specified by:

`add` in interface `Queue<E>`

Overrides:

`add` in class `AbstractCollection<E>`

Parameters:

`e` - element whose presence in this collection is to be ensured

Returns:

`true` (as specified by `Collection.add(E)`)

Throws:

`NullPointerException` - if the specified element is null

poll

```
public E poll()
```

Description copied from interface: Deque

Retrieves and removes the head of the queue represented by this deque (in other words, the first element of this deque), or returns `null` if this deque is empty.

This method is equivalent to `Deque.pollFirst()`.

Specified by:

`poll` in interface `Deque<E>`

Specified by:

`poll` in interface `Queue<E>`

Returns:

the first element of this deque, or null if this deque is empty

peek

```
public E peek()
```

Description copied from interface: Deque

Retrieves, but does not remove, the head of the queue represented by this deque (in other words, the first element of this deque), or returns `null` if this deque is empty.

This method is equivalent to `Deque.peekFirst()`.

Specified by:

peek in interface Deque<E>

Specified by:

peek in interface Queue<E>

Returns:

the head of the queue represented by this deque, or null if this deque is empty

remove

public E remove()

Description copied from interface: Deque

Retrieves and removes the head of the queue represented by this deque (in other words, the first element of this deque). This method differs from poll only in that it throws an exception if this deque is empty.

This method is equivalent to Deque.removeFirst().

Specified by:

remove in interface Deque<E>

Specified by:

remove in interface Queue<E>

Returns:

the head of the queue represented by this deque

Throws:

NoSuchElementException - if this deque is empty

pop

public E pop()

Description copied from interface: Deque

Pops an element from the stack represented by this deque. In other words, removes and returns the first element of this deque.

This method is equivalent to Deque.removeFirst().

Specified by:

pop in interface Deque<E>

Returns:

the element at the front of this deque (which is the top of the stack represented by this deque)

Throws:

NoSuchElementException - if this deque is empty

element

public E element()

Description copied from interface: Deque

Retrieves, but does not remove, the head of the queue represented by this deque (in other words, the first element of this deque). This method differs from peek only in that it throws an exception if this deque is empty.

This method is equivalent to Deque.getFirst().

Specified by:

element in interface Deque<E>

Specified by:

element in interface Queue<E>

Returns:

the head of the queue represented by this deque

Throws:

NoSuchElementException - if this deque is empty

push

public void push(E e)

Description copied from interface: Deque

Pushes an element onto the stack represented by this deque (in other words, at the head of this deque) if it is possible to do so immediately without violating capacity restrictions, throwing an IllegalStateException if no space is currently available.

This method is equivalent to Deque.addFirst(E).

Specified by:

push in interface Deque<E>

Parameters:

e - the element to push

Throws:

NullPointerException - if the specified element is null and this deque does not permit null elements

removeFirstOccurrence

```
public boolean removeFirstOccurrence(Object o)
```

Removes the first element `e` such that `o.equals(e)`, if such an element exists in this deque. If the deque does not contain the element, it is unchanged.

Specified by:

`removeFirstOccurrence` in interface `Deque<E>`

Parameters:

`o` - element to be removed from this deque, if present

Returns:

`true` if the deque contained the specified element

Throws:

`NullPointerException` - if the specified element is null

removeLastOccurrence

```
public boolean removeLastOccurrence(Object o)
```

Removes the last element `e` such that `o.equals(e)`, if such an element exists in this deque. If the deque does not contain the element, it is unchanged.

Specified by:

`removeLastOccurrence` in interface `Deque<E>`

Parameters:

`o` - element to be removed from this deque, if present

Returns:

`true` if the deque contained the specified element

Throws:

`NullPointerException` - if the specified element is null

contains

```
public boolean contains(Object o)
```

Returns `true` if this deque contains at least one element `e` such that `o.equals(e)`.

Specified by:

`contains` in interface `Collection<E>`

Specified by:

`contains` in interface `Deque<E>`

Overrides:

`contains` in class `AbstractCollection<E>`

Parameters:

`o` - element whose presence in this deque is to be tested

Returns:

`true` if this deque contains the specified element

isEmpty

```
public boolean isEmpty()
```

Returns `true` if this collection contains no elements.

Specified by:

`isEmpty` in interface `Collection<E>`

Overrides:

`isEmpty` in class `AbstractCollection<E>`

Returns:

`true` if this collection contains no elements

size

```
public int size()
```

Returns the number of elements in this deque. If this deque contains more than `Integer.MAX_VALUE` elements, it returns `Integer.MAX_VALUE`.

Beware that, unlike in most collections, this method is *NOT* a constant-time operation. Because of the asynchronous nature of these deques, determining the current number of elements requires traversing them all to count them. Additionally, it is possible for the size to change during execution of this method, in which case the returned result will be inaccurate. Thus, this method is typically not very useful in concurrent applications.

Specified by:

`size` in interface `Collection<E>`

Specified by:

`size` in interface `Deque<E>`

Specified by:

`size` in class `AbstractCollection<E>`

Returns:

the number of elements in this deque

remove

`public boolean remove(Object o)`

Removes the first element `e` such that `o.equals(e)`, if such an element exists in this deque. If the deque does not contain the element, it is unchanged.

Specified by:

`remove` in interface `Collection<E>`

Specified by:

`remove` in interface `Deque<E>`

Overrides:

`remove` in class `AbstractCollection<E>`

Parameters:

`o` - element to be removed from this deque, if present

Returns:

`true` if the deque contained the specified element

Throws:

`NullPointerException` - if the specified element is null

addAll

`public boolean addAll(Collection<? extends E> c)`

Appends all of the elements in the specified collection to the end of this deque, in the order that they are returned by the specified collection's iterator. Attempts to `addAll` of a deque to itself result in `IllegalArgumentException`.

Specified by:

`addAll` in interface `Collection<E>`

Overrides:

`addAll` in class `AbstractCollection<E>`

Parameters:

`c` - the elements to be inserted into this deque

Returns:

`true` if this deque changed as a result of the call

Throws:

`NullPointerException` - if the specified collection or any of its elements are null

`IllegalArgumentException` - if the collection is this deque

See Also:

`AbstractCollection.add(Object)`

clear

```
public void clear()
```

Removes all of the elements from this deque.

Specified by:

`clear` in interface `Collection<E>`

Overrides:

`clear` in class `AbstractCollection<E>`

toArray

```
public Object[] toArray()
```

Returns an array containing all of the elements in this deque, in proper sequence (from first to last element).

The returned array will be "safe" in that no references to it are maintained by this deque. (In other words, this method must allocate a new array). The caller is thus free to modify the returned array.

This method acts as bridge between array-based and collection-based APIs.

Specified by:

`toArray` in interface `Collection<E>`

Overrides:

`toArray` in class `AbstractCollection<E>`

Returns:

an array containing all of the elements in this deque

toArray

```
public <T> T[] toArray(T[] a)
```

Returns an array containing all of the elements in this deque, in proper sequence (from first to last element); the runtime type of the returned array is that of the specified array. If the deque fits in the specified array, it is returned therein. Otherwise, a new array is allocated with the runtime type of the specified array and the size of this deque.

If this deque fits in the specified array with room to spare (i.e., the array has more elements than this deque), the element in the array immediately following the end of the deque is set to `null`.

Like the `toArray()` method, this method acts as bridge between array-based and collection-based APIs. Further, this method allows precise control over the runtime type of the output array, and may, under certain circumstances, be used to save allocation costs.

Suppose `x` is a deque known to contain only strings. The following code can be used to dump the deque into a newly allocated array of `String`:

```
String[] y = x.toArray(new String[0]);
```

Note that `toArray(new Object[0])` is identical in function to `toArray()`.

Specified by:

`toArray` in interface `Collection<E>`

Overrides:

`toArray` in class `AbstractCollection<E>`

Type Parameters:

`T - the runtime type of the array to contain the collection`

Parameters:

`a - the array into which the elements of the deque are to be stored, if it is big enough; otherwise, a new array of the same runtime type is allocated for this purpose`

Returns:

`an array containing all of the elements in this deque`

Throws:

`ArrayStoreException - if the runtime type of the specified array is not a supertype of the runtime type of every element in this deque`

`NullPointerException - if the specified array is null`

iterator

```
public Iterator<E> iterator()
```

Returns an iterator over the elements in this deque in proper sequence. The elements will be returned in order from first (head) to last (tail).

The returned iterator is *weakly consistent*.

Specified by:

`iterator` in interface `Iterable<E>`

Specified by:

`iterator` in interface `Collection<E>`

Specified by:

`iterator` in interface `Deque<E>`

Specified by:

`iterator` in class `AbstractCollection<E>`

Returns:

an iterator over the elements in this deque in proper sequence

descendingIterator

```
public Iterator<E> descendingIterator()
```

Returns an iterator over the elements in this deque in reverse sequential order. The elements will be returned in order from last (tail) to first (head).

The returned iterator is *weakly consistent*.

Specified by:

`descendingIterator` in interface `Deque<E>`

Returns:

an iterator over the elements in this deque in reverse order

spliterator

```
public Spliterator<E> spliterator()
```

Returns a `Spliterator` over the elements in this deque.

The returned spliterator is *weakly consistent*.

The `Spliterator` reports `Spliterator.CONCURRENT`, `Spliterator.ORDERED`, and `Spliterator.NONNULL`.

Specified by:

`spliterator` in interface `Iterable<E>`

Specified by:

`spliterator` in interface `Collection<E>`

Implementation Note:

The Spliterator implements trySplit to permit limited parallelism.

Returns:

a Spliterator over the elements in this deque

Since:

1.8

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.